

```
"""
```

```
AI Usage vs. GPA – Statistical Simulation & Verification  
AP Statistics | Doral Academy | Prof. Kristina Zogović  
Replicates & verifies all statistical processes from Project 2
```

```
"""
```

```
import random  
import math  
  
try:  
    import numpy as np  
    import matplotlib.pyplot as plt  
    import matplotlib.patches as mpatches  
    from scipy import stats  
    SCIPY_AVAILABLE = True  
except ImportError:  
    SCIPY_AVAILABLE = False  
    print("⚠️  scipy/matplotlib not found – running in pure-Python mode.\n")
```

```
# =====  
# 0.  RAW DATA  (from Appendix, n = 50)  
#      Columns: Grade, GPA, AI_Minutes, AI_Days, AI_School_pct,  
#               Study_Hours, Advanced_Classes, Sleep_Hours,  
#               Activities (0/1), AI_Experience  
# =====
```

```
RAW = [  
    [10,2.65,86,2,82,6.4,1,6.4,0,"7-12 months"],  
    [11,3.12,19,6,71,10.6,5,6.7,1,"7-12 months"],  
    [12,3.52,8,3,53,7.5,5,6.5,0,"1 year+"],  
    [12,2.66,99,3,74,2.5,2,6.9,1,"7-12 months"],  
    [11,3.77,40,7,48,12,1,7.7,1,"1 year+"],  
    [10,3.38,36,5,86,10.4,2,8.1,0,"7-12 months"],  
    [10,2.79,33,6,76,11.7,3,7.3,1,"4-6 months"],  
    [12,3.29,22,3,76,11.3,1,6.6,0,"4-6 months"],  
    [9,2.58,99,4,70,10.5,5,8.3,1,"1 year+"],  
    [9,3.03,18,6,55,3.7,5,6.1,1,"1 year+"],  
    [12,2.65,91,6,90,6.9,2,7.6,0,"4-6 months"],  
    [10,2.85,99,5,70,4.1,3,8.1,0,"7-12 months"],  
    [10,3.5,74,3,91,6,2,6.1,0,"1 year+"],  
    [12,3.61,16,6,66,2.6,0,6.8,1,"1-3 months"],  
    [9,3.18,80,7,52,5.8,0,6.3,1,"7-12 months"],
```

```

[12,3.47,59,7,46,11.9,3,8.4,1,"7-12 months"],
[12,3.59,9,3,46,4.7,4,6.4,1,"4-6 months"],
[11,3.75,8,7,82,9.8,4,7.1,0,"1 year+"],
[11,3.45,16,4,67,6.6,0,7.8,0,"7-12 months"],
[9,3.97,32,5,62,6.2,0,7.4,0,"1 year+"],
[9,3.21,34,7,67,11.6,4,6.3,1,"1 year+"],
[11,3.54,69,7,66,12,1,8.4,0,"1 year+"],
[11,3.28,82,4,69,7.6,4,7.7,0,"4-6 months"],
[9,3.86,8,5,95,9.2,2,6.4,0,"1 year+"],
[11,3.73,76,6,86,3.5,1,6.1,0,"4-6 months"],
[12,3.26,30,5,43,5,2,6.9,1,"1-3 months"],
[10,2.96,96,2,83,11.7,0,7.4,1,"7-12 months"],
[12,3.03,88,3,81,7.8,1,7.1,0,"7-12 months"],
[9,3.11,94,3,81,7.4,2,6.1,1,"1-3 months"],
[11,3.03,74,2,46,9.5,2,6.9,0,"4-6 months"],
[10,2.52,58,4,43,2.6,1,8.3,1,"7-12 months"],
[9,4,33,2,65,7.8,3,8.4,1,"4-6 months"],
[11,3,62,6,86,7,4,6.1,0,"4-6 months"],
[10,2.85,80,6,61,10.5,5,6.9,0,"4-6 months"],
[10,3.13,40,3,91,3.6,2,7.7,0,"1-3 months"],
[11,2.94,5,6,95,11.6,4,7.7,0,"1-3 months"],
[10,3.54,25,3,46,2.8,5,6.9,1,"4-6 months"],
[9,2.88,94,2,55,3.9,4,7.4,0,"1 year+"],
[11,3.64,59,2,52,8,0,8.2,1,"1-3 months"],
[12,3.44,48,7,52,8.8,5,8.4,0,"1 year+"],
[9,3.39,40,7,74,4.4,4,7.9,1,"1 year+"],
[9,3.75,24,2,68,3.2,2,8.3,0,"4-6 months"],
[11,4,32,3,48,10.9,5,6.6,1,"1 year+"],
[11,3.11,48,2,67,4.5,0,6.4,1,"1 year+"],
[10,3.32,18,2,51,7.9,1,8,0,"1 year+"],
[9,4,16,4,57,8.2,2,6.4,0,"4-6 months"],
[10,2.7,53,2,69,6.2,0,7,0,"4-6 months"],
[9,4,17,6,55,7.8,0,6.4,0,"1-3 months"],
[9,3.09,50,3,95,7.2,5,8.3,0,"1-3 months"],
]

# =====
# SECTION 1 – UNIVARIATE DESCRIPTIVE STATISTICS (AP Units 1-2)
# =====

def descriptive_stats(values, label):
    n = len(values)
    mu = sum(values) / n

```

```

var = sum((x - mu)**2 for x in values) / (n - 1)
sd = math.sqrt(var)
s = sorted(values)
def percentile(p):
    idx = (p/100) * (n - 1)
    lo, hi = int(idx), min(int(idx)+1, n-1)
    return s[lo] + (idx - lo) * (s[hi] - s[lo])
q1, med, q3 = percentile(25), percentile(50), percentile(75)
iqr = q3 - q1
print(f"\n{'-'*55}")
print(f"  {label}  (n={n})")
print(f"{'-'*55}")
print(f"  Mean                : {mu:.4f}")
print(f"  Std Dev (s)          : {sd:.4f}")
print(f"  Min                  : {min(values):.4f}")
print(f"  Q1                   : {q1:.4f}")
print(f"  Median               : {med:.4f}")
print(f"  Q3                   : {q3:.4f}")
print(f"  Max                  : {max(values):.4f}")
print(f"  IQR                  : {iqr:.4f}")
lo_fence = q1 - 1.5 * iqr
hi_fence = q3 + 1.5 * iqr
outliers = [x for x in values if x < lo_fence or x > hi_fence]
print(f"  Outliers (1.5IQR) : {outliers if outliers else 'None'}")
return {"mean": mu, "sd": sd, "min": min(values), "q1": q1,
        "median": med, "q3": q3, "max": max(values), "iqr": iqr}

```

```

# =====
# SECTION 2 – BIVARIATE: CORRELATION & LSRL (AP Units 2–3)
# =====

```

```

def correlation_and_regression(x_vals, y_vals):
    n = len(x_vals)
    x_mu = sum(x_vals) / n
    y_mu = sum(y_vals) / n
    sx = math.sqrt(sum((xi - x_mu)**2 for xi in x_vals) / (n-1))
    sy = math.sqrt(sum((yi - y_mu)**2 for yi in y_vals) / (n-1))
    r = sum((x_vals[i]-x_mu)*(y_vals[i]-y_mu) for i in range(n)) /
    ((n-1)*sx*sy)
    b = r * (sy / sx)          # slope
    a = y_mu - b * x_mu        # intercept
    y_hat = [a + b * xi for xi in x_vals]
    residuals = [y_vals[i] - y_hat[i] for i in range(n)]

```

```

sse = sum(r_**2 for r_ in residuals)
s   = math.sqrt(sse / (n - 2))    # residual std dev
r2  = r**2
print(f"\n{'-'*55}")
print(" BIVARIATE ANALYSIS: AI Hours → GPA")
print(f"{'-'*55}")
print(f" Pearson r          : {r:.4f}")
print(f" R²                  : {r2:.4f} ({r2*100:.1f}% of variance
explained)")
print(f" Slope (b)           : {b:.4f}")
print(f" Intercept (a)        : {a:.4f}")
print(f" LSRL                :  $\hat{y} = {a:.3f} + ({b:.3f})x$ ")
print(f" Residual Std Dev     : {s:.4f}")
return {"r": r, "b": b, "a": a, "s": s, "r2": r2,
        "residuals": residuals, "y_hat": y_hat}

# =====
# SECTION 3 – INFERENCE: t-TEST FOR SLOPE  $\beta$  (AP Units 6–9)
# =====

def t_test_for_slope(x_vals, reg, alpha=0.05):
    n = len(x_vals)
    x_mu = sum(x_vals) / n
    ssx = sum((xi - x_mu)**2 for xi in x_vals)
    se_b = reg["s"] / math.sqrt(ssx)
    t = reg["b"] / se_b
    df = n - 2
    # Two-tailed p-value approximation via t-distribution CDF
    # Uses scipy if available, otherwise a quick numerical approximation
    if SCIPY_AVAILABLE:
        p_val = stats.t.sf(abs(t), df) * 2    # two-tailed; report one-tailed
    for Ha:  $\beta < 0$ 
        p_one = stats.t.sf(abs(t), df)        # one-tailed (Ha:  $\beta < 0$ )
    else:
        # Simple normal approximation for large df
        z = abs(t)
        p_one = 0.5 * math.erfc(z / math.sqrt(2))
        p_val = 2 * p_one
    print(f"\n{'-'*55}")
    print(" INFERENCE: t-Test for Slope ( $H_0: \beta = 0$ ,  $H_a: \beta < 0$ )")
    print(f"{'-'*55}")
    print(f" SE of slope (SEb) : {se_b:.4f}")
    print(f" t-statistic       : {t:.4f}")

```

```

print(f" Degrees of freedom: {df}")
print(f" p-value (one-tail): {p_one:.8f}")
print(f" p-value (two-tail): {p_val:.8f}")
print(f"  $\alpha$  : {alpha}")
decision = "REJECT H0" if p_one < alpha else "FAIL TO REJECT H0"
print(f" Decision : {decision}")
if decision == "REJECT H0":
    print("  $\checkmark$  Significant negative linear relationship detected.")
return {"t": t, "df": df, "se_b": se_b, "p_one": p_one, "p_two": p_val}

# =====
# SECTION 4 – CONFIDENCE INTERVAL FOR  $\beta$  (AP Unit 8)
# =====

def confidence_interval_slope(reg, inf, confidence=0.95):
    df = inf["df"]
    alpha_two = 1 - confidence
    if SCIPY_AVAILABLE:
        t_crit = stats.t.ppf(1 - alpha_two/2, df)
    else:
        # Critical value approximation for large df ( $\geq 30$ )
        t_crit = 2.011 # df=48 @ 95% (tabulated)
    margin = t_crit * inf["se_b"]
    lo = reg["b"] - margin
    hi = reg["b"] + margin
    print(f"\n{'-'*55}")
    print(f" {int(confidence*100)}% Confidence Interval for  $\beta$  (slope)")
    print(f"{'-'*55}")
    print(f" t* (df={df}) : {t_crit:.4f}")
    print(f" Margin of Error :  $\pm$ {margin:.4f}")
    print(f" CI : ({lo:.4f}, {hi:.4f})")
    print(f" Interpretation : We are {int(confidence*100)}% confident the
true slope")
    print(f" lies between {lo:.4f} and {hi:.4f} GPA pts per hour.")

# =====
# SECTION 5 – STRATIFIED SAMPLING SIMULATION (AP Unit 3)
# =====

def stratified_sampling_demo(population_size=4000, sample_size=50, seed=42):
    random.seed(seed)
    grades = [9, 10, 11, 12]
    strata_size = population_size // len(grades) # 1000 each

```

```

per_stratum = sample_size // len(grades)          # 12-13 each
sample_ids = []
print(f"\n{'-'*55}")
print("  STRATIFIED RANDOM SAMPLING SIMULATION")
print(f"{'-'*55}")
for i, grade in enumerate(grades):
    n_select = per_stratum + (1 if i < (sample_size % len(grades)) else 0)
    ids = random.sample(range(1 + i*strata_size, 1 + (i+1)*strata_size),
n_select)
    sample_ids.extend(ids)
    print(f"  Grade {grade}: selected {n_select} of {strata_size} students
→ IDs sample: {ids[:4]}...")
    print(f"  Total sampled : {len(sample_ids)}")
    print(f"  10% condition : {len(sample_ids)} < {population_size*0.1:.0f}
√")

```

```

# =====
# SECTION 6 – BOOTSTRAP CONFIDENCE INTERVAL  (AP Unit 8 extension)
# =====

```

```

def bootstrap_slope_ci(x_vals, y_vals, n_boot=5000, confidence=0.95, seed=0):
    random.seed(seed)
    n = len(x_vals)
    slopes = []
    for _ in range(n_boot):
        idx = [random.randint(0, n-1) for _ in range(n)]
        xs = [x_vals[i] for i in idx]
        ys = [y_vals[i] for i in idx]
        xm = sum(xs)/n; ym = sum(ys)/n
        num = sum((xs[i]-xm)*(ys[i]-ym) for i in range(n))
        den = sum((xs[i]-xm)**2 for i in range(n))
        if den != 0:
            slopes.append(num/den)
    slopes.sort()
    lo_idx = int((1-confidence)/2 * len(slopes))
    hi_idx = int((1-(1-confidence)/2) * len(slopes)) - 1
    print(f"\n{'-'*55}")
    print(f"  BOOTSTRAP CI for slope  ({n_boot} resamples)")
    print(f"{'-'*55}")
    print(f"  Bootstrap mean slope : {sum(slopes)/len(slopes):.4f}")
    print(f"  {int(confidence*100)}% Bootstrap CI   : ({slopes[lo_idx]:.4f},
{slopes[hi_idx]:.4f})")

```

```
# =====  
# SECTION 7 – CATEGORICAL ANALYSIS (AP Unit 1)  
# =====
```

```
def categorical_analysis(data):  
    print(f"\n{'-'*55}")  
    print(" CATEGORICAL VARIABLE ANALYSIS")  
    print(f"{'-'*55}")  
    # AI Experience distribution  
    exp_counts = {}  
    act_gpa = {"Yes": [], "No": []}  
    for row in data:  
        exp = row[9]  
        exp_counts[exp] = exp_counts.get(exp, 0) + 1  
        act_gpa["Yes" if row[8] else "No"].append(row[1])  
    print("\n AI Experience Levels:")  
    total = sum(exp_counts.values())  
    for k, v in sorted(exp_counts.items()):  
        print(f" {k:15s} : {v:2d} ({v/total*100:.0f}%)")  
    print("\n GPA by Extracurricular Participation:")  
    for grp, gpas in act_gpa.items():  
        mean = sum(gpas)/len(gpas)  
        print(f" {grp:3s} ({len(gpas):2d} students) : mean GPA =  
{mean:.3f}")
```

```
# =====  
# SECTION 8 – MATPLOTLIB PLOTS (if scipy/matplotlib available)  
# =====
```

```
def generate_plots(x_vals, y_vals, reg):  
    if not SCIPY_AVAILABLE:  
        print("\n⚠ Plots skipped (matplotlib not available).")  
        return  
    fig, axes = plt.subplots(2, 2, figsize=(12, 9))  
    fig.suptitle("AI Usage vs GPA – Statistical Dashboard\n"  
                "AP Statistics | Doral Academy", fontsize=13,  
                fontweight="bold")  
  
    # 1. Scatterplot + LSRL  
    ax = axes[0, 0]  
    ax.scatter(x_vals, y_vals, color="#2563EB", alpha=0.7, edgecolors="white",  
s=60)  
    xs_line = [min(x_vals), max(x_vals)]
```

```

ys_line = [reg["a"] + reg["b"]*xi for xi in xs_line]
ax.plot(xs_line, ys_line, color="#DC2626", lw=2, label=f" $\hat{y} =$ 
{reg['a']:.3f} + ({reg['b']:.3f})x")
ax.set_xlabel("Daily AI Usage (hours)"); ax.set_ylabel("GPA")
ax.set_title(f"Scatterplot + LSRL (r = {reg['r']:.3f})")
ax.legend(fontsize=8); ax.grid(alpha=0.3)

# 2. Residual plot
ax = axes[0, 1]
ax.scatter(reg["y_hat"], reg["residuals"], color="#7C3AED", alpha=0.7,
edgecolors="white", s=55)
ax.axhline(0, color="#DC2626", lw=1.5, linestyle="--")
ax.set_xlabel("Fitted Values ( $\hat{y}$ )"); ax.set_ylabel("Residuals")
ax.set_title("Residual Plot"); ax.grid(alpha=0.3)

# 3. GPA histogram
ax = axes[1, 0]
ax.hist(y_vals, bins=14, color="#059669", edgecolor="white", alpha=0.85)
ax.axvline(sum(y_vals)/len(y_vals), color="#DC2626", lw=2, linestyle="--",
label=f"Mean = {sum(y_vals)/len(y_vals):.3f}")
ax.set_xlabel("GPA"); ax.set_ylabel("Frequency")
ax.set_title("GPA Distribution"); ax.legend(fontsize=8);
ax.grid(alpha=0.3)

# 4. AI Hours histogram
ax = axes[1, 1]
ax.hist(x_vals, bins=12, color="#D97706", edgecolor="white", alpha=0.85)
ax.axvline(sum(x_vals)/len(x_vals), color="#DC2626", lw=2, linestyle="--",
label=f"Mean = {sum(x_vals)/len(x_vals):.3f} hrs")
ax.set_xlabel("Daily AI Usage (hours)"); ax.set_ylabel("Frequency")
ax.set_title("AI Usage Distribution (right-skewed)");
ax.legend(fontsize=8); ax.grid(alpha=0.3)

plt.tight_layout()
plt.savefig("ai_gpa_plots.png", dpi=150, bbox_inches="tight")
plt.show()

# =====
# MAIN
# =====

def main():
    print("=" * 55)

```



```

print(" AI USAGE vs. GPA - STATISTICAL SIMULATION")
print(" AP Statistics | Doral Academy | 2025-2026")
print("=" * 55)

# --- Extract variables ---
gpa      = [row[1] for row in RAW]
ai_min   = [row[2] for row in RAW]
ai_hours = [m / 60 for m in ai_min] # convert minutes → hours
sleep    = [row[7] for row in RAW]
study    = [row[5] for row in RAW]

# UNIT 1-2: Descriptive stats
stats_gpa = descriptive_stats(gpa, "GPA (Response Variable y)")
stats_ai  = descriptive_stats(ai_hours, "AI Usage in Hours (Explanatory
Variable x)")
descriptive_stats(sleep, "Sleep Hours")
descriptive_stats(study, "Weekly Study Hours")

# UNIT 2-3: Regression & correlation
reg = correlation_and_regression(ai_hours, gpa)

# UNIT 6-9: Inference
inf = t_test_for_slope(ai_hours, reg)
confidence_interval_slope(reg, inf)

# UNIT 3: Sampling simulation
stratified_sampling_demo()

# UNIT 8: Bootstrap CI
bootstrap_slope_ci(ai_hours, gpa)

# UNIT 1: Categorical
categorical_analysis(RAW)

# Plots
generate_plots(ai_hours, gpa, reg)

print(f"\n{'='*55}")
print(" SUMMARY - KEY FINDINGS")
print(f"{'='*55}")
print(f"  r  = {reg['r']:.3f} → moderate negative linear association")
print(f"  R² = {reg['r2']:.3f} → AI hours explains {reg['r2']*100:.1f}%
of GPA variance")

```

```
print(f"  $\hat{y}$  = {reg['a']:.3f} + ({reg['b']:.3f}) × AI_hours")
print(f" p ≈ {inf['p_one']:.2e} → overwhelming evidence against  $H_0$ ")
print(f" ∴ Reject  $H_0$ ; significant negative relationship exists.\n")

if __name__ == "__main__":
    main()
```